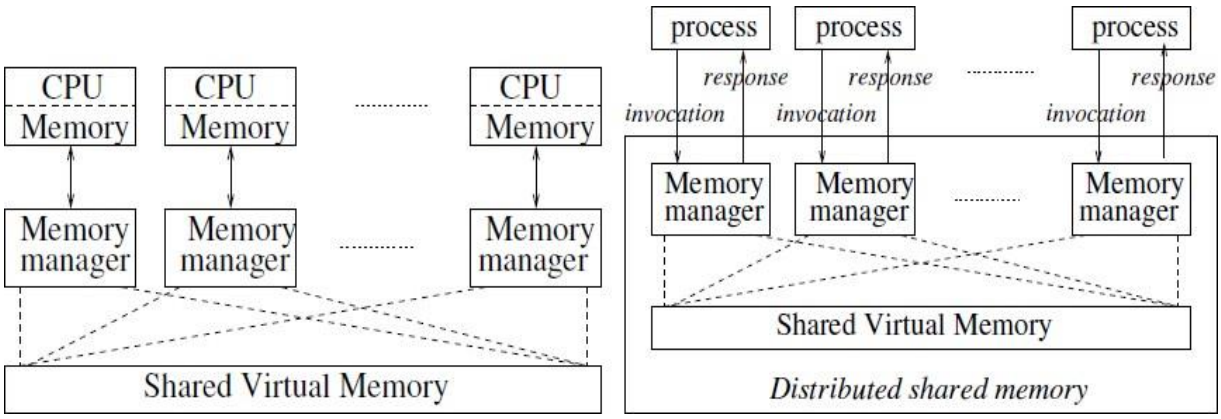
LECTURE 1

Introduction:

In computer science, **distributed shared memory (DSM)** is a form of memory architecture where physically separated memories can be addressed as one logically shared address space. A distributed shared memory system implements the shared-memory model on a physically distributed memory system.

- communicate with Read/Write ops in shared virtual space
- No Send and Receive primitives to be used by application
- Under covers, Send and Receive used by DSM manager
- Locking is too restrictive; need concurrent access
- With replica management, problem of consistency arises
- weaker consistency models (weaker than von Neumann)

Non-uniform memory access (NUMA) is a computer memory design used in multiprocessing, where the memory access time depends on the memory location relative to the processor. Under NUMA, a processor can access its own local memory faster than non-local memory (memory local to another processor or memory shared between processors).



Advantages/Disadvantages of DSM

Advantages:

- Shields programmer from Send/Receive primitives
- Single address space; simpli_es passing-by-reference and passing complex data structures
- Exploit locality-of-reference when a block is moved
- DSM uses simpler software interfaces, and cheaper o_-the-shelf hardware. Hence cheaper than dedicated multiprocessor systems
- No memory access bottleneck, as no single bus
- Large virtual memory space
- DSM programs portable as they use common DSM programming interface

Disadvantages:

- Programmers need to understand consistency models, to write correct programs
- DSM implementations use async message-passing, and hence cannot be more
- e_cient than msg-passing implementations
- By yielding control to DSM manager software, programmers cannot use their own
- Msg-passing solutions.

Issues in Implementing DSM Software

- Semantics for concurrent access must be clearly speci_ed
- Semantics {replication? partial? full? read-only? write-only?
- Locations for replication (for optimization)
- If not full replication, determine location of nearest data for access
- Reduce delays, # msgs to implement the semantics of concurrent access

LECTURE 2

MEMORY CONSISTENCY MODEL:

A memory consistency model for a shared memory multiprocessor specifies hence, memory behaves with respect to read and write operation from multiple processors.

A consistency model is the definition of when modifications to data may be seen at a given processor. It defines hence memory will appear to a programmer by placing restrictions on the values. That can be returned by a read of memory locations.

A consistency model is essential a contract between the software and the memory. It says that if the software agrees to obey certain, all set are off and correctness of memory operation in no longer guarantee.

Types of consistency:-

1. **Strict consistency:-**any read to memory location X return value stored by the most recent writes are instantaneously visible to all processes and an absolute global time order is maintained.

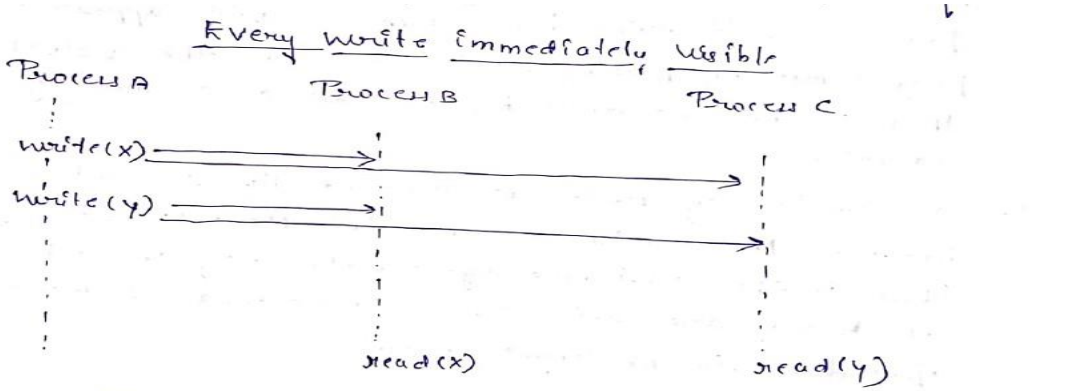


Fig:- Every write is immediate visible in strict consistency

2. **Sequential consistency:-**In this, the result of any execution is the same as if the operation of all processor were executed in some sequential order, and the operation of each individual processor appear is. This sequence in order specified by its program.
3. **Casual consistency:-**writes that the potentially casually related must be seen by all process in the same order. Concurrent writes may be seen in a different order on different machine.

Multiprocessor Cache Systems :⇒

↳ Cache Coherence refers to the Consistency of data store in local caches of a shared resource.

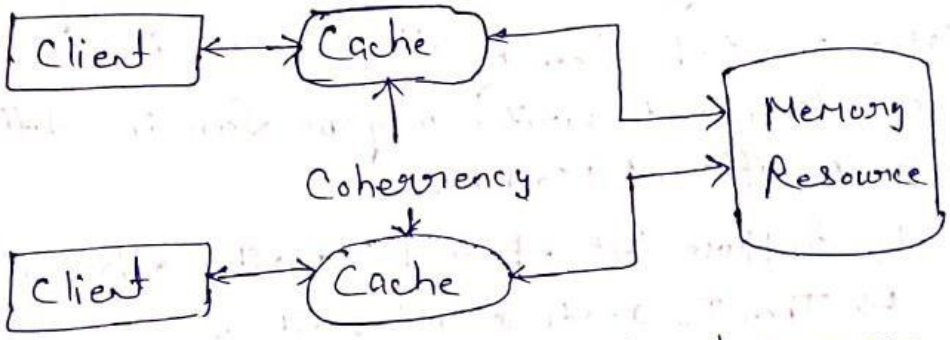


Fig 1 – Multiple caches of shared resource

- Cache coherence is intended to manage such continue and maintain consistency between cache and memory.

Cache coherence protocols:-

- A coherency protocol is a protocol which maintains the consistency between all the caches in a system of distributed shared memory.
- The protocol maintains memory coherence according to a specific consistency model.
- Cache coherence protocol is classified as write Invalid or writes update.

LECTURE 3

Distributed system memory

In distributed shared memory, our goal is to simulate a collection ofmemory locations or registers, each of which supports a read operationthat returns the current state of the register and a write operation thatupdates the state. Our implementation should be linearizable,meaning that read and write operations appear to occur instantaneously(atomically) at some point in between when the operation starts and theoperation finishes; equivalently, there should be some way to order all theoperations on the registers to obtain a sequential execution consistentwith the behavior of a real register (each read returns the value of the mostrecent write) while preserving the observable partial order on operations(where _1 precedes _2 if _1 finishes before _2 starts). Implicit in this definition

is the assumption that implemented operations take place over some interval,between an invocation that starts the operation and a response that endsthe operation and returns its value.

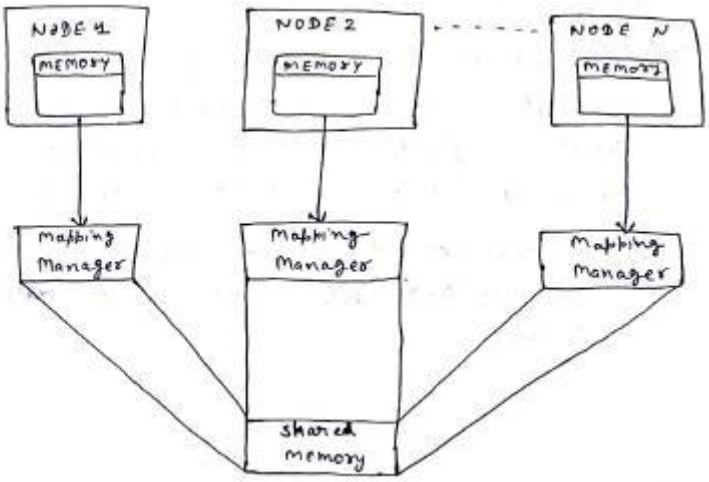


Fig:- Distributed Shared Memory

Models of distributed computation

Central server algorithm:-

1. In this a central server algorithm maintain all the shared data. A remote access can be implemented as a request sent to a centralized server, and the server respond with returned data centralized for reads and an acknowledgement for writes.
2. It service the read request from other nodes or clientized by returning the data itmes to them.
3. It updates the data on write request by client and returns acknowledgment messages.
4. In these strategy should blocked is neither migrated nor replicated.

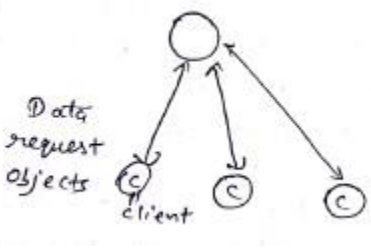


Fig:- Client server model

Migration Algorithm:-

1. Data in migration algorithm is shipped to the location of data access request, allowing subsequent access to the data to be performed locally.
2. The migration algorithm allows only one node to access a shared data at time.
3. The migration algorithm can make use of a server that keeps tracks of the location of pager or through links maintained at nodes.

Read replication algorithm:-

1. It is extends the migration algorithm by replicating data blocks and allowing multiple nodes to have read access or one node to have read write access.
2. It can improve system performance by allowing multiple node to access data concurrently.

Full replication Algorithm:-

1. It is an extension of read replication algorithm.

2. It allows multiple nodes to have both read and write access to shared data block, because many nodes can write shared data concurrently.
3. In this, all node wishing to modify should data will send the modification to sequencer. The sequence will assign a sequence number and multicast the modification with the sequence number to all the nodes that have a copy of shared data item.

LECTURE 4

A DISTRIBUTED system is a collection of autonomous computing nodes which can communicate with eachother and which cooperate on a common goal or task [4]. Forexample, the goal may be to provide the user with a databasemanagement system, and in this case the distributed systemis called a distributed database [16].When a node fails or when the communication subsystemwhich allows nodes to communicate fails, it is usually necessaryfor the nodes to adapt to the new conditions so that they maycontinue working on their joint goal. For example, considera collection of nodes which are processing sensory data andtrying to locate a moving object [18]. Each node has somesensors which provide it with a local view of the world. Thenodes exchange data and together decide where the object islocated. If one of the nodes ceases to operate, the remainingnodes should recognize this and modify their strategy for locatingthe object. A node which neighbors the failed node couldtry to collect sensory data for the area which was assigned tothe failed node. Another alternative would be for the remainingnodes to use a detection algorithm which is not very sensitiveto "holes" in the area being studied. Or the nodes could decideto switch to such an algorithm when the failure occurs. Ifenough nodes fail, the remaining nodes may decide that theyjust cannot perform the assigned task, and may select a newor better suited job for themselves.

Dynamic load sharing and balancing:

The assumption of prior knowledge of processes is not realistic for most distributed applications. The disjoint process model, which ignores the effect of the interdependency among processes, is used. Objective of scheduling: *utilization* of the system (has direct bearing on throughput and completion time) and *fairness* to the user processes (difficult to define).

If we can designate a controller process that maintains the information about the queue size of each processor:

- Fairness in terms of equal workload on each processor (join the shortest queue) – migration workstation model (use of **load sharing** and **load balancing**, perhaps **load redistribution** i.e. **process migration**)
- Fairness in terms of user's share of computation resources (allocate processor to a waiting process at a user site that has the least share of the processor pool) – processor pool model

Solutions without a centralized controller: **sender-** and **receiver-initiated** algorithms.

Sender-initiated algorithm:

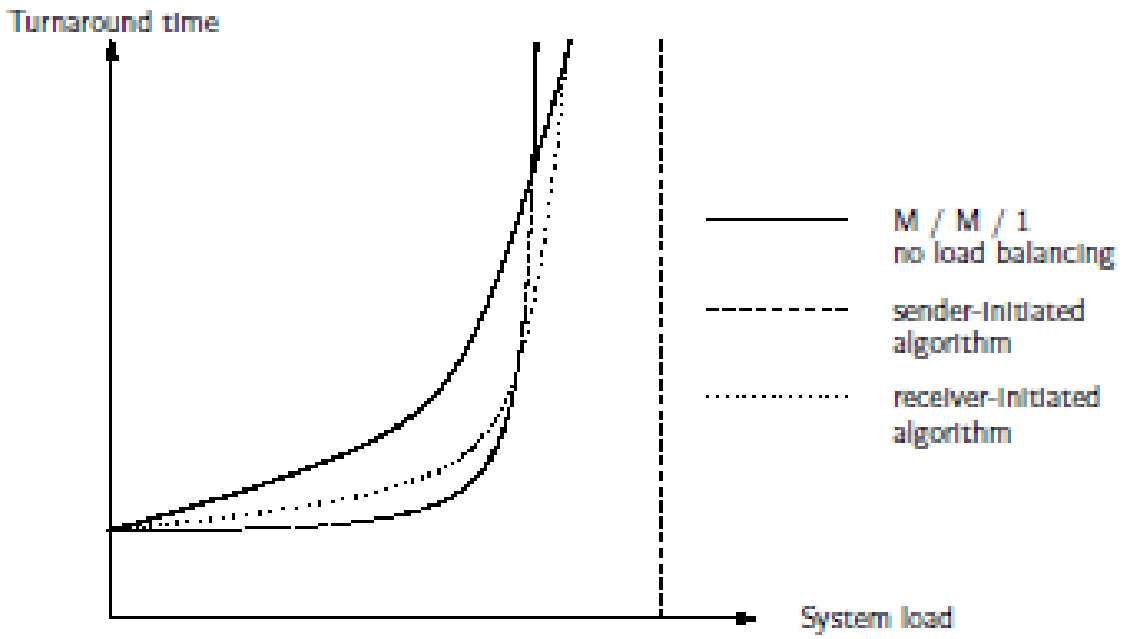
- Push model

- Includes probing strategy for finding a node with the smallest queue length(perhaps multicast)
- Performs well on a lightly loaded system

Receiver-initiated algorithm:

- Pull model
- Probing strategy can also be used
- More stable
- Perform on average better

Combinations of both algorithms are possible: choice based on the estimated system load information or reaching threshold values of the processing node's queue.



Performance comparison of dynamic load-sharing algorithms

Distributed file system

File system - storage for persistent named data objects (files); responsible for creation, deletion, retrieval, modification and protection of files.

Distributed file system (DFS) is an implementation of a file system that consists of physically dispersed storage sites but which provide a traditional file system view for users (files crossing system boundaries is transparent to the users). Many important concepts in distributed systems design can be demonstrated by the implementation of a DFS:

1. DFS employ many aspects of transparency
2. Directory service in a DFS is a good example of a name service
3. Use of caching and replication → problem of cache coherence and replica management

4. Access control and protection → distributed system security problems

Characteristics of a DFS:

Two key characteristics: *dispersion* and *multiplicity* of users and files. Goal: hiding this from users. A transparent DFS should exhibit the following transparency properties:

- **Dispersed clients:** *login transparency* and *access transparency* (regardless of whether files are on the local or remote host)
- **Dispersed files:** names used to refer to files need not contain information about their physical location – *location transparency*, *location independence* (can be moved from one location to another without changing their names)
- **Multiplicity of users:** *concurrency transparency*, desirable to support at the transaction level
- **Multiplicity of files:** *replication transparency* (atomic updates on the replicas)

Other desirable characteristics: fault tolerance, scalability, heterogeneity of the system.

DFS design and implementation:

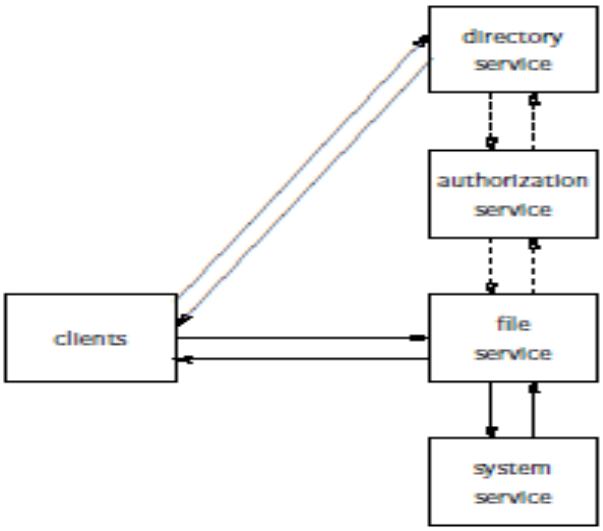
To the users, a file consists of: **file name**, **file attributes**, **data units**. Organization of data units: *flat* or *hierarchical*. File accesses:

- *sequential access*: file position is maintained with each open file – used in a *session* of subsequent file accesses
- *direct access*: explicit reference fixed-size data units by their block number
- *indexed sequential access*: data units addresses directly using an index (key) associated with each data block

Major components of a file system:

directory service		name resolution, addition and deletion of files
authorization service		capability and/or access control list
file service	transaction	concurrency and replication management
	basic	read/write files and get/set attributes
system service		device, cache, and block management

Services in a DFS ⟶ usually structured to provide directory, authorization, file and system services separately.



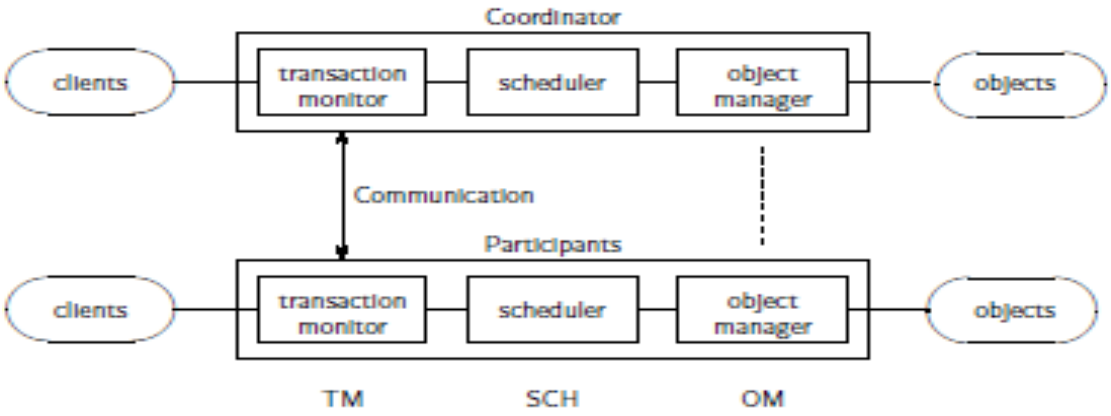
File mounting and server registration

Directories are hierarchically structured. Mounting operation: attaches a remote named file system to the client's file system hierarchy at the position pointed to by a path name (usually a leaf of the directory tree that contains only an empty subdirectory). Linkage information (*mount table*) for the remote FS is maintained by the host FS. A file server normally restricts

LECTURE 5

Transaction and concurrency control:

Simple architecture for a distributed transaction processing system:



Atomicity is the primary goal of atomic transactions. Isolation and durability are secondary in the sense that they are needed only to support atomicity.

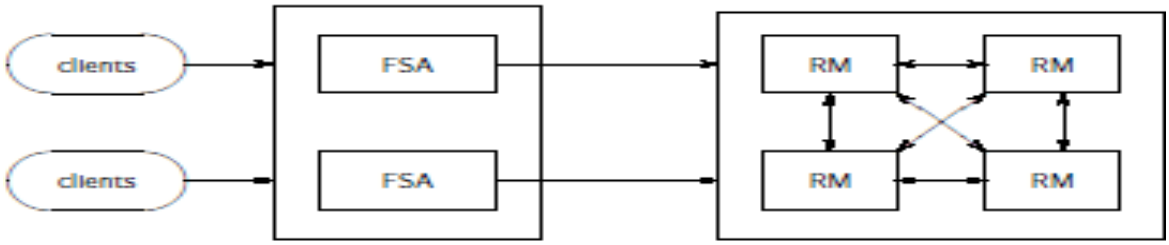
Optimistic concurrency control:

- Allowing an entire transaction to complete and the validate the transaction before making its effect permanent
- Assumes a private workspace that contains shadow copies of the shared objects at each transaction site
- In the execution phase no validation of consistency is performed
- On end transaction the TM begins the validating phase – a transaction is globally validated among TMs using a two-phase commit protocol
- After the consistency of the transaction is validated, the TM enters the update phase to make changes permanent in the persistent memory.
- Once a transaction is validated, it is guaranteed to be committed.

Data and file replication

Updates to all replicas must be *atomic*. **One-copy serializability** for distributed transaction services: combined atomic transaction and atomic update property.

Architecture for management of replicas



Object access operations: *read*, *update* (*overwrite*, *read-modify*).

Options for read operations:

- read-one-primary
- read-one
- Read-quorum

Options for write operations:

- Write-one-primary
- Write-all
- Write-all-available
- Write-quorum

LECTURE 6

CASE STUDY

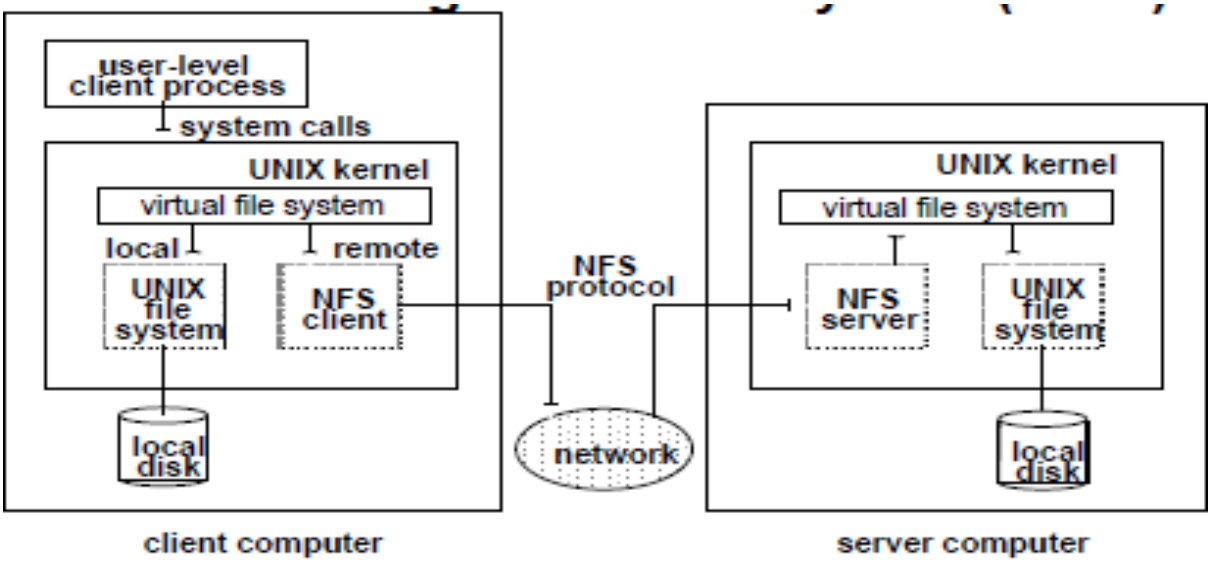
1. NFS(SUN)
2. GPFS
3. GENERAL FILE SYSTEM (ANDREW AND CODA FILE)

NFS(SUN)

Sun's network file systems (NFS)

- Designed by Sun Microsystems
  - First distributed file service designed as a project, introduced in 1985
  - To encourage its adoption as a standard
    - Definitions of the key interfaces were placed in the public domain in 1989
    - Source code for a reference implementation was made available to other computer vendors under license
    - Currently the de facto standard for LANs
- Provides transparent access to remote files on a LAN, for clients running on UNIX and other operating systems
  - A UNIX computer typically has a NFS client and server module in its OS kernel
    - Available for almost any UNIX
    - Client modules are available for Macintosh and PCs

- Remote file systems may be
  - Hard mounted — when a user-level process accesses a file, it is suspended until the request can be completed
    - If a server crashes, the user-level process will be suspended until recovers
  - Soft mounted — after a small number of retries, the NFS client returns a failure code to the user process
    - Most UNIX utilities don't check this code…
- Automounting
  - The automounter dynamically mounts a file system whenever an "empty" mount point is referenced by a client
    - Further accesses do not result in further requests to the automounter…
    - Unless there are no references to the remote file system for several minutes, in which case the automounter unmounts it



- Virtual file system:
  - Separates generic file-system operations from their implementation (can have different types of local file systems)
  - Based on a file descriptor called a vnode that is unique networkwide (UNIX inodes are only unique on a single file system)

LECTURE 7

NFS protocol

- NFS protocol provides a set of RPCs for remote file operations
  - Looking up a file within a directory
  - Manipulating links and directories
  - Creating, renaming, and removing files
  - Getting and setting file attributes
  - Reading and writing files
- NFS is stateless
  - Servers do not maintain information about their clients from one access to the next
    - There are no open-file tables on the server
  - There are no open and close operations
    - Each request must provide a unique file identifier, and an offset within the file
  - Easy to recover from a crash, but file operations must be idempotent

- Because NFS is stateless, all modified data must be written to the server's disk before results are returned to the client
  - Server crash and recovery should be invisible to client —data should be intact
  - Lose benefits of caching
    - Solution — RAM disks with battery backup (un-interruptable power supply), written to disk periodically
- A single NFS write is guaranteed to be atomic, and not intermixed with other writes to the same file
  - However, NFS does not provide concurrency control
    - A `write` system call may be decomposed into several NFS writes, which may be interleaved
    - Since NFS is stateless, this is not considered to be an NFS problem

Andrew file system (AFS)

- Designed by Carnegie Mellon University
  - Developed during mid-1980s as part of the Andrew distributed computing environment
  - Designed to support a WAN of more than 5000 workstations
  - Much of the core technology is now part of the Open Software Foundation (OSF) Distributed Computing Environment (DCE), available for most UNIX and some other operating systems
- AFS was made to span large campuses and scale well therefore the emphasis was placed on offloading the work to the clients
- as much as possible data is cached on clients, uses session semantics - cache consistency operations are done when file is opened or closed
- Provides transparent access to remote files on a WAN, for clients running on UNIX and other operating systems
  - Access to all files is via the usual UNIX file primitives
  - Compatible with NFS — servers can mount NFS file systems

- AFS is stateful, when a client reads a file from a server it holds a callback, the server keeps track of callbacks and when one of the clients closes the file (and synchronizes it's cached copy) and updates it, the server notifies all the callback holders of the change breaking the callback, callbacks can be also broken to conserve storage at server
- problems with AFS:
  - even if the data is in local cache - if the client performs a write a complex protocol of local callback verification with the server must be used; cache consistency preservation leads to deadlocks
  - in a stateful model, it is hard to deal with crushes.

LECTURE 8

Caching in Andrew

- When a remote file is accessed, the server sends the entire file to the client
  - The entire file is then stored in a disk cache on the client computer
    - Cache is big enough to store several hundred files
- Implements session semantics
  - Files are cached when opened
  - Modified files are flushed to the server when they are closed
  - Writes may not be immediately visible to other processes
- When client caches a file, server records that fact — it has a callback on the file
  - When a client modifies and closes a file, other clients lose their callback, and are notified by server that their copy is invalid

How can Andrew perform well?

- Most file accesses are to files that are infrequently updated, or are accessed by only a single user, so the cached copy will remain valid for a long time
- Local cache can be big — maybe 100 MB — which is probably sufficient for one user's working set of files
- Typical UNIX workloads:
  - Files are small, most are less than 10kB
  - Read operations are 6 times more common than write operations
  - Sequential access is common, while random access is rare
  - Most files are read and written by only one user; if a file shared, usually only one user modifies it
  - Files are referenced in bursts

AFS vs. NFS

- NFS is simpler to implement, AFS is cumbersome due to cache consistency synchronization mechanisms, etc
- NFS does not scale well - it is usually limited to a LAN, AFS scales well - it may span the Internet
- NFS performs better than AFS under light to medium load. AFS does better under heavy load
- NFS does writes faster(?)

what is the available operation on AFS (ANDREW FILE SYSTEM)?

ANS:-

1. LOOKUP:- ALLOWS A USER TO LIST THE CONTENT OF THE AFS DIRECTORY.
2. INSERT:- ALLOW A USER TO ADD NEW FILE OR SUBDIRECTORIES TO THE DIRECTORY.
3. DELETE:-ALLOW A USER TO DELETE FILE OR SUBDIRECTORIES FROM THE DIRECTORY.
4. ADMINISTRATER:- ALLOW A USER TO CHANGE THE PERMISSION OF THE AFS DIRECTORY(HOME).
5. READ:-ALLOW A USER TO LOOK AT THE CONTENTS OF FILE AND SUBDIRECTORIES TO THE AFS DIRECTORIES, UNTIL READ ACCESS TO ANY USER.
6. WRITE:-ALLOW A USER TO MODIFY FILES IN A DIRECTORY.

HOW DOES AFS WORK?

CODA (CONSTANT DATA AVAILABILTY) FILE SYSTEM:

Coda is a file system for a large-scale distributedcomputing environment composed of Unix workstations. It providesresiliency to server and network failures through the use oftwo distinct but complementary mechanisms.

1. One mechanism, ***server replication***, stores copies of a file at multiple servers.

2. The other mechanism, ***disconnected operation***, is a mode of execution in which a caching site temporarily assumes the role of a replication site. Disconnected operation is particularly useful for supporting portable workstations. The design of Coda optimizes for availability and performance, and strives to provide the highest degree of consistency attainable in the light of these objectives. Measurements from a prototype show that the performance cost of providing high availability in Coda is reasonable.

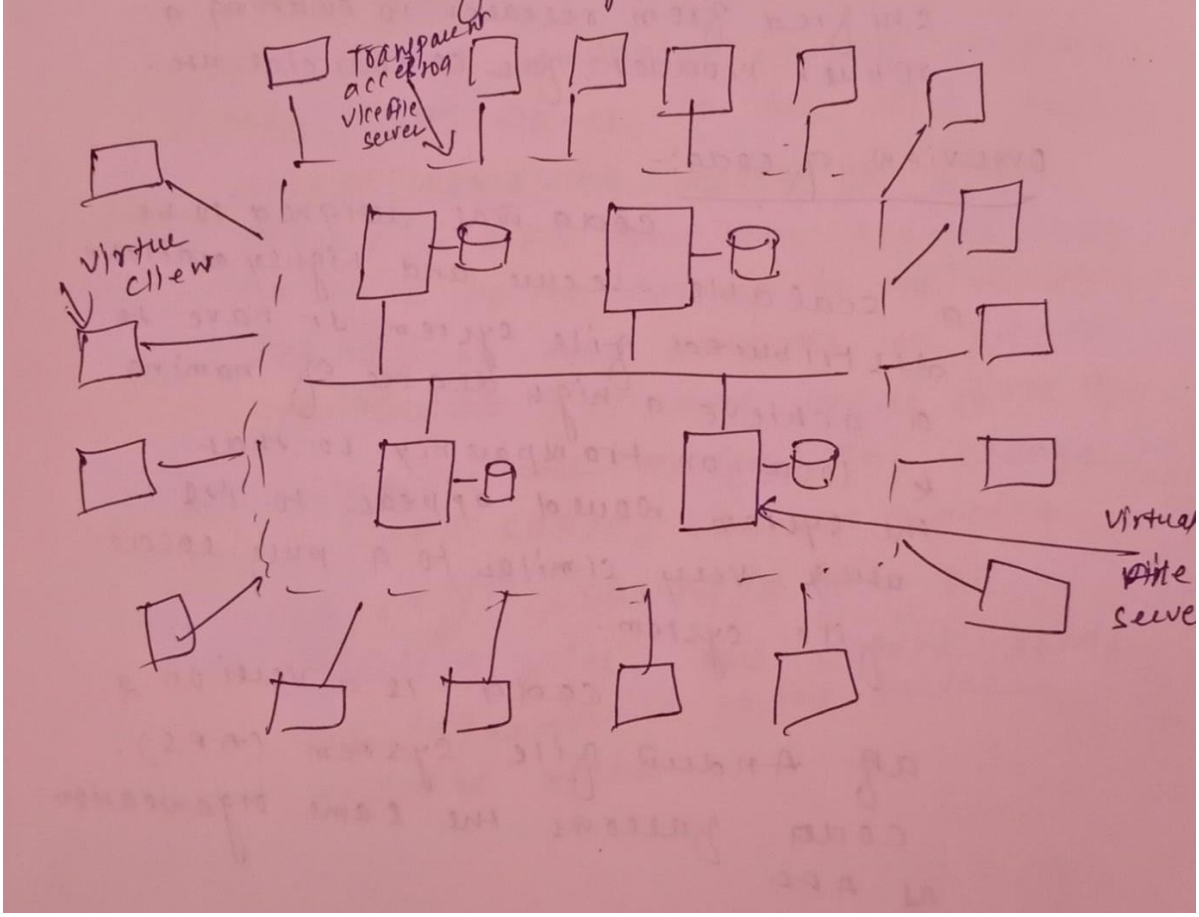
3. IT WAS DESIGNED TO BE A SCALABLE, SECURE, AND HIGHLY AVAILABLE DISTRIBUTED FILE SYSTEM.

4. CODA IS A 2ND VERSON OF ANDREW FILE SYSTEM(AFS) WITH THE SAME ORGANISATION.

The *Coda File System* is a descendant of AFS that is substantiallymore resilient to failures. The ideal that Coda strivesfor is *constant data availability*, allowing a user to continueworking regardless of failures elsewhere in the system. Ourgoal is to provide users with the benefits of a

shared datarepository, but to allow them to rely entirely on local resourceswhen that repository is partially or totally inaccessible.A related goal of Coda is to gracefully integrate the useof AFS with portable computers. At present, users manuallycopy relevant files from AFS, use the machine while isolatedfrom the network, and manually copy updated files back toAFS upon reconnection. These users are effectively performingmanual caching of files with write-back on reconnection.If one views the disconnection from AFS as a deliberatelyinducedfailure, it is clear that a mechanism for supportingportable machines in isolation is also a mechanism for faulttolerance.

→ Coda was designed to be scalable, secure; and highly available distributed file system. An important goal was to achieve a high degree of naming and location transparency so that system would appear to its use very similar to a pure local file system



FEATURES OF CODA FILE SYSTEM:-

1. THROUGH THE SERVER REPLICATION DATA AVAILABILITY IS INCREASED.
2. SECURITY MODEL FOR AUTHENTICATION, ENCRYPTION AND ACCESS CONTROL
3. GOOD SCALABILITY
4. HIGH PERFORMANCE THROUGH THE CLIENT SIDE FROM THE CACHE CONSISTENCY
5. CONTINUE OPERATION DURING PARTIAL NETWORK FAILURES IN SERVER NETWORK.
6. SHARING IS WELL DEFINED IN THE PRESENCE OF NETWORK FAILURE.

- FOR THE TRANSPARENCY THE SYSTEM WOULD APPEAR TO ITS USER TO A PURE LOCAL FILE SYATEM.
- FEATURES OF CODA FILE SYSTEM
    1. HIGH PERFORMANCE THROUGH THE CLIENT SIDE
    2. SERVER REPLICATION (THROUGH THE REPLICATION MANAGER)
    3. IN THIS AUTHENTICATION ENCRYPTION AND ACCESS CONTROL IS USED.

4. CONTINUE OPERATION DURING PARTIAL NETWORK FAILURE IN SERVER NETWOK(BECAUSE OF SERVER REPLICATION)
5. GOOD SCALABILITY
6. DURING PARTIAL NETWORK FAILURE WELL DEFINED FOR SHARING OF FILE ETC.

Features :-

- Disconnected operation for mobile computing.
- Is freely available under a liberal licence
- High performance through client side persistent caching.
- Server replication
- Security model for authentication, encryption and access control.
- Continued operation during partial network failures in server n/w.
- Good scalability
- Well defined semantics of sharing, even in the presence of n/w failures